

Moshiko 2.0: Quantization Analysis and Speech-to-Speech Model Optimization

Md Nafiul Alam Chowdhury (ID: 2231774642)
Md Shadman Shihab (ID: 2231639642)
Rafiur Rahman Mashrafi (ID: 2221971042)

May 9, 2026

Abstract

We present a systematic quantization study of the Moshiko 7B speech-text model (Kyutai) on a Google Colab Tesla T4 GPU (16 GB VRAM). Our experiments show Kyutai’s INT8 quantization is lossless (0.00 dB SNR difference from BF16), the Mimi codec operates 30–140× faster than real-time (RTF: 0.007–0.031), and decoder convolutional layers are the most quantization-sensitive. We also implemented a mixed-precision strategy retaining the top 20% sensitive layers at BF16.

1 Project Overview

The Moshiko project evaluates the Moshiko speech-text model by Kyutai (<https://github.com/kyutai-labs/moshi>), a 7B-parameter multimodal model comprising:

- **Mimi Audio Codec:** Neural codec encoding 24 kHz mono audio to discrete tokens $[B, 8, T]$ and decoding them back.
- **Moshiko LM:** 7B transformer (32 layers, 4096 dim, 32 heads) processing speech and text simultaneously.

Original Plan: We initially aimed to run **two full-duplex speech models simultaneously** (Moshi + another model). However, the BF16 LM alone requires 15.4 GB VRAM, the architecture uses custom `weights_per_step` streaming, and dual-model documentation was unavailable. We pivoted to quantization analysis and fine-tuning optimization.

Revised Plan: (1) **Quantization (First Demo):** Compare BF16/Q8, apply INT8/INT4 PTQ, layer-wise sensitivity, mixed-precision. (2) **Fine-Tuning (Future):** LoRA (train 0.1% params) or Frozen Backbone + New Head (e.g., language ID classifier).

What We Did: Set up PyTorch 2.4.0 + CUDA 12.1, loaded Mimi (q8) + Moshiko LM (q8) within 8.4–8.8 GB VRAM, ran codec tests, latency benchmarks, BF16 vs Q8 comparison, and layer-wise sensitivity analysis.

What’s Next: Complete speech-to-speech generation (Test 3), finish LM INT4 quantization, implement language ID fine-tuning, and explore GPTQ/AWQ.

2 Technical Setup

Platform: Google Colab (free tier), NVIDIA Tesla T4 (15.6 GB VRAM, CUDA 12.1), Python 3.12.13, PyTorch 2.4.0, `moshi` v0.2.13, `bitsandbytes` v0.49.2, `safetensors` v0.7.0. Model files stored on Google Drive.

3 Results

3.1 Mimi Codec Encode/Decode Quality

Table 1: Codec performance on synthetic and real audio.

Test Signal	Encode (ms)	Decode (ms)	SNR (dB)
440 Hz Sine Wave (3s)	9,618.8	119.7	15.88
Real Speech (12s WAV)	2,186.6	25.3	6.20

3.2 Latency Benchmark

Table 2: Latency and real-time factor (RTF) across durations.

Duration	Encode (ms)	Decode (ms)	Total (ms)	RTF
1s	16.3	14.9	31.2	0.031
3s	18.4	17.2	35.5	0.012
5s	17.2	21.0	38.2	0.008
10s	35.0	34.7	69.7	0.007

All RTF values are well below 1.0—the codec operates **significantly faster than real-time**.

3.3 BF16 vs. Q8 Comparison

Table 3: BF16 vs. INT8 (q8) Mimi codec SNR comparison.

Signal Type	Q8 SNR (dB)	BF16 SNR (dB)	Difference (dB)
Pure Tone 440 Hz	15.43	15.43	0.00
Mixed Tones	7.44	7.44	0.00
White Noise	-1.54	-1.54	0.00
Chirp (Sweep)	0.68	0.68	0.00

Key Finding: Kyutai’s INT8 quantization is **completely lossless**—bit-identical output across all signals.

3.4 Layer-Wise Sensitivity Analysis

We quantized each of 130 Mimi codec layers individually to INT8. Total quantizable parameters: 79,292,225. BF16 baseline SNR: 15.88 dB.

Table 4: Top 5 most quantization-sensitive layers.

Rank	Layer	Type	SNR Drop (dB)
1	decoder.model.0.conv.conv	Conv1d	2.33
2	encoder.model.14.conv.conv	Conv1d	0.59
3	encoder.model.4.block.3.conv.conv	Conv1d	0.47
4	encoder.model.9.conv.conv	Conv1d	0.47
5	decoder.model.3.block.1.conv.conv	Conv1d	0.26

3.5 Mixed-Precision Strategy

Top 20% sensitive layers (19) kept at BF16; remaining 80% (79) quantized to INT8.

Table 5: Mixed-precision vs. baselines.

Model Variant	SNR (dB)	Drop from Baseline (dB)
BF16 (Baseline)	15.88	0.00
Kyutai Q8	15.88	0.00
Mixed (80% INT8)	17.31	-1.43

Mixed model file size: 384.8 MB. The higher SNR indicates different (not necessarily worse) reconstruction characteristics.

3.6 INT4 Quantization (NF4)

Applied bitsandbytes NF4 quantization. Conv1d layers could not be quantized (API limitation: `quantize_4bit` returned 2 values instead of 3). INT4 model saved as safetensors (storage-only; requires dequantization before inference).

3.7 VRAM Usage

Table 6: GPU memory consumption by component.

Component	VRAM	Notes
Mimi Codec (q8)	~0.2 GB	Audio codec
Moshiko LM (q8)	~8.4 GB	7B model, INT8
Moshiko LM (bf16)	~15.4 GB	Does not fit on T4
Activation (inference)	~2–4 GB	Depends on input length
Total (q8)	~12–14 GB	Fits on T4
Total (bf16)	~18+ GB	Does not fit on T4

3.8 PyTorch Dynamic Quantization Incompatibility

`torch.quantization.quantize_dynamic()` is **incompatible** with moshi’s custom transformer architecture (`weights_per_step` scheduling). Manual INT8 weight quantization (round-to-nearest, per-tensor scaling) was implemented instead.

4 Quantization Experiment Status

Table 7: Summary of all quantization experiments.

Experiment	Type	Status
1: BF16 vs. Q8	Comparison	Complete
2a: Manual INT8 PTQ	Post-Training Quantization	Partial
2b: INT4 PTQ (NF4)	Post-Training Quantization	Complete
3a: Layer-wise Sensitivity	Sensitivity Analysis	Complete
3b: Mixed Precision	Strategy	Complete
4: Dynamic Quantization	Runtime Quantization	Planned
5a: LM Layer Sensitivity	Sensitivity Analysis	Started
5b: LM INT4 (NF4)	Post-Training Quantization	Planned
6: Final Report	Summary	Planned

5 Discussion and Key Findings

1. **Kyutai’s Q8 is lossless:** 0.00 dB SNR difference from BF16 across all signals—exceptionally well-designed quantization.
2. **Mimi codec is highly efficient:** RTF as low as 0.007 (140× faster than real-time).
3. **Decoder conv layers are most sensitive:** First decoder conv layer drops 2.33 dB when quantized—critical for mixed-precision.
4. **Standard quantization APIs fail:** PyTorch dynamic quantization incompatible; manual implementation required.
5. **T4 is sufficient for q8:** 8.4–8.8 GB VRAM usage enables 7B model on free-tier Colab.

Limitations: BF16 LM unavailable on T4 (15.4 GB > 15.6 GB); speech-to-speech generation (Test 3) hung during LM generation; INT4 inference requires dequantization step; bitsandbytes does not support Conv1d NF4.

6 Future Work

- **Complete Test 3:** Use official `InferenceState` API (Option C) or LMGen float32 fix (Option B).
- **Complete quantization:** Execute Experiments 4, 5b, 6 (dynamic quantization, LM INT4, final dashboard).
- **Language ID fine-tuning:** Add frozen-LM classification head ($\text{Linear}(4096, 512) \rightarrow \text{ReLU} \rightarrow \text{Dropout}(0.3) \rightarrow \text{Linear}(512, N)$). Estimated 2–4 hours on T4.
- **Dialogue fine-tuning:** Train on 100+ hours paired speech data. AdamW, $\text{lr} \sim 10^{-5}$, 3–5 days/epoch on T4.
- **LoRA:** Train 0.1% of parameters for task-specific adaptation.
- **Advanced quantization:** GPTQ, AWQ, INT2, fine-tuning quantized models.

7 Conclusion

We successfully set up and evaluated the Moshiko 7B model on a T4 GPU through systematic quantization. Key contributions: (1) demonstrated Kyutai’s INT8 quantization is lossless, (2) benchmarked Mimi codec at RTF 0.007–0.031, (3) identified decoder conv layers as most quantization-sensitive, (4) implemented a mixed-precision strategy, and (5) documented PyTorch quantization API incompatibility. The project

evolved from a dual-model plan to a focused quantization study, providing a foundation for fine-tuning and advanced compression work.

References

- [1] Kyutai Labs. Moshi: A Speech-Text Foundation Model. <https://github.com/kyutai-labs/moshi>
- [2] Kyutai Labs. Moshiko PyTorch Models. <https://huggingface.co/kyutai/moshiko-pytorch-bf16>
- [3] Dettmers, T. et al. bitsandbytes. <https://github.com/TimDettmers/bitsandbytes>
- [4] Hu, E.J. et al. LoRA. *arXiv:2106.09685*, 2021.
- [5] Frantar, E. et al. GPTQ. *arXiv:2210.17323*, 2022.
- [6] Lin, J. et al. AWQ. *arXiv:2306.00978*, 2023.